

Summer school ISIMA - OTH Phase 2

Mathematical Modeling and Optimization with Julia

Prof. Dr. Viet Hung Nguyen - ISIMA - Clermont Auvergne INP

Lesson 4 - Branch-and-Bound algorithm with Bonobo.jl for Integer Programming

August 2026

Binary knapsack model

$$\max \quad 7x_1 + 10x_2 + 4x_3 + 4x_4 + 8x_5$$

$$3x_1 + 5x_2 + 12x_3 + 2x_4 + 6x_5 \leq 15, \quad x_i \in \{0, 1\}.$$

item i	1	2	3	4	5
profit p_i	7	10	4	4	8
weight w_i	3	5	12	2	6

$x_i = 1$ if item i is selected, $x_i = 0$ otherwise.

LP relaxation at the root

The binary restrictions are relaxed:

$$x_i \in \{0, 1\} \quad \rightsquigarrow \quad 0 \leq x_i \leq 1.$$

The root relaxation gives

$$x_1 = x_2 = x_4 = 1, \quad x_3 = 0, \quad x_5 = \frac{5}{6}.$$

$$z_{LP} = 7 + 10 + 4 + 8 \cdot \frac{5}{6} = 27.67.$$

The solution is not binary, but it gives a valid upper bound:

$$z_{MIP}^* \leq 27.67.$$

The root node

root node
 $UB = 27.67$
 $x_1 = x_2 = x_4 = 1$
 $x_3 = 0, x_5 = 0.83$



The bound is available, but no incumbent is obtained:

$$x_5 = 0.83 \notin \{0, 1\}.$$

A fractional variable must be selected for branching.

Branching on a fractional variable

Branch on x_5 :

$$x_5 = 0 \quad \text{or} \quad x_5 = 1.$$

For a general integer variable with relaxation value x_j^* ,

$$x_j \leq \lfloor x_j^* \rfloor \quad \text{or} \quad x_j \geq \lceil x_j^* \rceil.$$

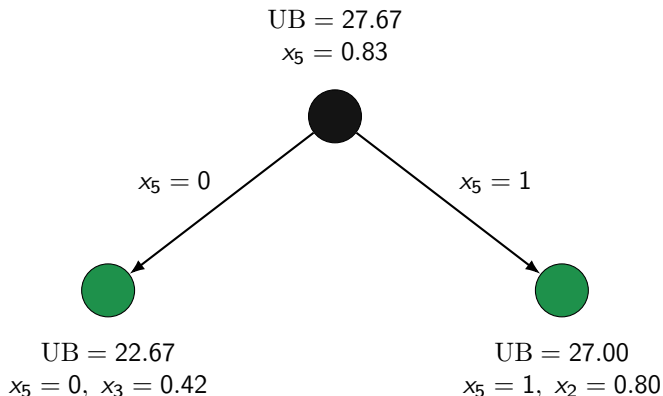
Each child is a restriction of the parent node:

$$P_{\text{child}} \subseteq P_{\text{parent}}.$$

For maximization,

$$\text{UB}(\text{child}) \leq \text{UB}(\text{parent}).$$

First branching step



The right child has the larger upper bound and remains fractional.

Global bounds

For a maximization problem, Branch-and-Bound maintains

LB = value of the best integer solution found so far,

and

$$UB = \max\{UB(N) : N \in \mathcal{O}\},$$

where $\mathcal{O}$ is the set of open nodes.

At every iteration,

$$LB \leq z^* \leq UB.$$

The optimality gap can be measured by

$$UB - LB \quad \text{or} \quad \frac{UB - LB}{|LB|}.$$

Updating the global lower bound

A new lower bound is obtained only from an integer feasible solution.

If a node gives an integer solution $\bar{x}$, compute

$$z(\bar{x}) = c^T \bar{x}.$$

Then

$$LB \leftarrow \max\{LB, z(\bar{x})\}.$$

If $z(\bar{x}) > LB$, then $\bar{x}$ becomes the incumbent.

For the knapsack solution

$$x = (1, 1, 0, 0, 1),$$

we obtain

$$LB = 7 + 10 + 8 = 25.$$

Updating the global upper bound

Each open node has a local upper bound.

After processing a node:

$\mathcal{O} \leftarrow$ open nodes remaining after fathoming or branching.

Then

$$UB \leftarrow \max\{UB(N) : N \in \mathcal{O}\}.$$

If $\mathcal{O} = \emptyset$, the search is complete.

$$z^* = LB.$$

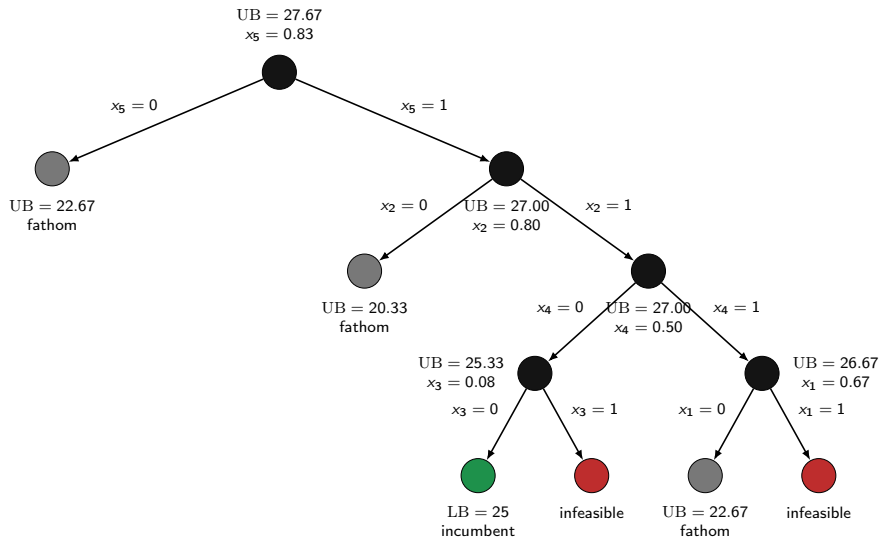
If $UB \leq LB$, no open node can improve the incumbent.

Fathoming rules

A node N is fathomed when it cannot produce a better integer solution.

Case	Action	Justification
Relaxation infeasible	fathom N	no feasible descendant
Relaxation integer	update LB, fathom N	feasible integer solution
$UB(N) \leq LB$	fathom N	bound dominated by incumbent
Fractional and promising	branch N	descendants may improve LB

Complete search tree for the example



Enumeration avoided by fathoming

A complete binary tree with N binary variables has

$$2^{N+1} - 1$$

nodes.

For $N = 5$:

$$2^6 - 1 = 63.$$

Branch-and-Bound avoids full enumeration by combining:

relaxation bounds + incumbent updates + fathoming.

Generic Branch-and-Bound scheme

1. $\mathcal{O} \leftarrow \{\text{root}\}, \quad \text{LB} \leftarrow -\infty.$
2. Evaluate root and compute UB.
3. **while** $\mathcal{O} \neq \emptyset$ **and** $\text{UB} > \text{LB}$:
4. select a node $N \in \mathcal{O}.$
5. remove N from $\mathcal{O}.$
6. compute a valid bound for $N.$
7. fathom, update incumbent, or branch.
8. insert children when branching occurs.
9. $\text{UB} \leftarrow \max\{\text{UB}(Q) : Q \in \mathcal{O}\}.$

A numerical implementation usually stops when the gap is below a tolerance.

Role of Bonobo

Bonobo manages the tree structure and delegates problem-specific operations.

Branch-and-Bound object	Bonobo implementation
node	subtype of <code>BB.AbstractNode</code>
node evaluation	<code>BB.evaluate_node!</code>
branching candidates	<code>BB.get_branching_indices</code>
children	<code>BB.get_branching_nodes_info</code>
node selection	<code>traverse_strategy</code>
incumbent	stored in the tree state
search log	callback

Node evaluation is not necessarily LP-based

Bonobo is a generic Branch-and-Bound framework.

Evaluating a node means computing a valid bound for the corresponding subproblem.

Possible bounding mechanisms include:

- LP relaxation;
- Lagrangian relaxation;
- convex relaxation;
- combinatorial bound;
- greedy upper bound for knapsack.

In the MILP implementation below, the bound is obtained from the LP relaxation solved by JuMP and HiGHS.

Packages and data

```
using Bonobo
using JuMP
using HiGHS

import MathOptInterface as MOI

const BB = Bonobo

profit = [7.0, 10.0, 4.0, 4.0, 8.0]
weight = [3.0, 5.0, 12.0, 2.0, 6.0]
capacity = 15.0
n = length(profit)
```

The following implementation uses the same knapsack data as the previous B&B tree.

LP relaxation in JuMP: variables and capacity

```
function build_lp_relaxation()
    model = Model(HiGHS.Optimizer)
    set_silent(model)

    @variable(model, 0 <= x[1:n] <= 1)

    @constraint(model,
        sum(weight[i] * x[i] for i in 1:n) <= capacity
    )

    return model, x
end
```

The model represents the continuous relaxation of the binary knapsack.

LP relaxation in JuMP: objective

```
function build_lp_relaxation()
    model = Model(HiGHS.Optimizer)
    set_silent(model)

    @variable(model, 0 <= x[1:n] <= 1)

    @constraint(model,
        sum(weight[i] * x[i] for i in 1:n) <= capacity
    )

    @objective(model, Max,
        sum(profit[i] * x[i] for i in 1:n)
    )

    return model, x
end
```

At a B&B node, additional bounds will be imposed on these variables.

Node representation

A node is represented by variable bounds:

$$\ell_i \leq x_i \leq u_i.$$

Root node:

$$\ell_i = 0, \quad u_i = 1.$$

After branching on x_5 :

$$u_5 = 0 \quad \text{or} \quad \ell_5 = 1.$$

This avoids creating a new JuMP model for every node.

Custom Bonobo node

```
mutable struct KnapsackNode <: BB.AbstractNode
    std::BB.BnBNodeInfo

    lbs::Vector{Float64}
    ubs::Vector{Float64}

    status::MOI.TerminationStatusCode

    parent_id::Int
    branch_text::String

    lp_objective::Float64
    lp_solution::Vector{Float64}
    integral::Bool
end
```

The field `std` is required by Bonobo. The remaining fields describe the current subproblem and its relaxation solution.

Branching candidates

All variables are binary in the knapsack model.

```
function BB.get_branching_indices(model::JuMP.Model)
    return collect(1:num_variables(model))
end
```

In a mixed-integer model, the function should return only the integer or binary variables.

Node evaluation: impose node bounds

```
function BB.evaluate_node!(tree, node::KnapsackNode)
    model = tree.root
    vars = all_variables(model)

    for i in eachindex(vars)
        set_lower_bound(vars[i], node.lbs[i])
        set_upper_bound(vars[i], node.ubs[i])
    end

    optimize!(model)

    # continued on the next slide
end
```

The same relaxation model is reused, but with the bounds of the current node.

Node evaluation: solve and store the relaxation

```
node.status = termination_status(model)

if node.status != MOI.OPTIMAL
    return NaN, NaN
end

node.lp_objective = objective_value(model)
node.lp_solution = value.(vars)
```

For a maximization problem, `node.lp_objective` is a local upper bound for the current node.

Node evaluation: integer test and return values

```
node.integral = all(  
    BB.is_approx_feasible.(tree, node.lp_solution)  
)  
  
if node.integral  
    return node.lp_objective, node.lp_objective  
else  
    return node.lp_objective, NaN  
end
```

Returned pair:

(node bound, incumbent candidate).

A fractional relaxation gives a bound but no incumbent candidate.

Relaxed solution for branching

```
function BB.get_relaxed_values(tree, node::KnapsackNode)
    return copy(node.lp_solution)
end
```

Bonobo uses these values to select a branching variable.

For binary variables, MOST_INFEASIBLE chooses a variable maximizing

$$\min(x_i^*, 1 - x_i^*).$$

Creating the left child

If x_j^* is fractional, the left child imposes

$$x_j \leq \lfloor x_j^* \rfloor.$$

```
function BB.get_branching_nodes_info(tree, node, vidx)
    val = node.lp_solution[vidx]
    children = NamedTuple[]

    left_ubs = copy(node.ubs)
    left_ubs[vidx] = floor(Int, val)

    push!(children, (
        lbs = copy(node.lbs),
        ubs = left_ubs,
        parent_id = node.std.id,
        branch_text = "x$(vidx) <= $(floor(Int, val))",
        lp_objective = NaN,
        lp_solution = fill(NaN, n),
        integral = false,
    ))
```

Creating the right child

The right child imposes

$$x_j \geq \lceil x_j^* \rceil.$$

```
right_lbs = copy(node.lbs)
right_lbs[vidx] = ceil(Int, val)

push!(children, (
    lbs = right_lbs,
    ubs = copy(node.ubs),
    parent_id = node.std.id,
    branch_text = "x$(vidx) >= $(ceil(Int, val))",
    lp_objective = NaN,
    lp_solution = fill(NaN, n),
    integral = false,
))

return children
end
```

Initialize the Bonobo tree

```
model, x = build_lp_relaxation()

tree = BB.initialize(
    branch_strategy = BB.MOST_INFEASIBLE(),
    traverse_strategy = BB.BestFirstSearch(),
    Node = KnapsackNode,
    root = model,
    sense = :Max,
)
```

The branching strategy chooses the variable.

The traversal strategy chooses the next open node.

Set the root node

```
BB.set_root!(  
    tree,  
    (  
        lbs = zeros(n),  
        ubs = ones(n),  
        status = MOI.OPTIMIZE_NOT_CALLED,  
        parent_id = 0,  
        branch_text = "root",  
        lp_objective = NaN,  
        lp_solution = fill(NaN, n),  
        integral = false,  
    ),  
)
```

The root contains no branching restriction beyond $0 \leq x_i \leq 1$.

Callback: classify the processed node

```
log = NamedTuple[]

callback = function(tree, node; kwargs...)
    fathom_reason =
        get(kwargs, :node_infeasible, false) ? :infeasible :
        get(kwargs, :worse_than_incumbent, false) ? :bound :
        node.integral ? :integer :
        :branched

    # log entry on the next slide
end
```

The status is aligned with the mathematical B&B cases: infeasible, bound, integer, or branched.

Callback: store the node information

```
push!(log, (  
    id = node.std.id,  
    parent = node.parent_id,  
    depth = node.std.depth,  
    restriction = node.branch_text,  
    status = fathom_reason,  
    lp_bound = node.lp_objective,  
    solution = copy(node.lp_solution),  
))
```

The log records the explored tree and the local upper bound at each node.

Run the search and extract the result

```
BB.optimize!(tree; callback = callback)

objective = BB.get_objective_value(tree)
solution = BB.get_solution(tree)
```

For the knapsack example:

$$x^* = (1, 1, 0, 0, 1), \quad z^* = 25.$$

The direct MIP solve with HiGHS gives the same result, but the internal tree is not explicitly exposed in the model code.

1. Define data.
2. Define a relaxation or another bounding method.
3. Define a custom node type.
4. Declare branching variables.
5. Implement node evaluation.
6. Return values used by the branching rule.
7. Create child-node information.
8. Initialize the Bonobo tree.
9. Set the root node.
10. Run the search.
11. Extract the incumbent and the search log.

Branch-and-Bound combines three mechanisms:

branching bounding fathoming.

The global lower bound is the incumbent value:

LB = best integer solution found so far.

The global upper bound is obtained from open nodes:

$$UB = \max\{UB(N) : N \in \mathcal{O}\}.$$

Bonobo supplies the tree-management framework. The bounding method is defined by the user.

Richard Lusby and Thomas Stidsen, *Mathematical Programming with Julia*, Section 5.2, “Solution algorithm: Branch & Bound”.

Bonobo.jl documentation:

<https://wikunia.github.io/Bonobo.jl/>

Bonobo.jl repository:

<https://github.com/Wikunia/Bonobo.jl>

JuMP documentation:

<https://jump.dev/JuMP.jl/stable/>